

**LAPORAN AKHIR
PROYEK TEKNOLOGI SERVER
PERANCANGAN SISTEM *EARLY WARNING* GANGGUAN INFRASTRUKTUR
BANK BERBASIS IOT**



Disusun Oleh:

Nama (NIM)	:	Nur Halimah	(24/540349/SV/24822)
		Tanaya Nindya Pradipta	(24/536017/SV/24365)
		Insyania Cahya Wiguna	(24/533789/SV/23938)
		Annas Meilya Khodijah	(24/544888/SV/25557)
Kelas	:	RI4AA	
Dosen Pengampu	:	Ardhi Wicaksono Santoso., S.Kom., M.Cs.	

**SARJANA TERAPAN TEKNOLOGI REKAYASA INTERNET
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
A. PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan.....	2
1.4 Manfaat.....	3
1.5 Batasan Masalah.....	3
B. TINJAUAN PUSTAKA	4
2.1 Teknologi Server	4
2.2 Virtual Private Server (VPS).....	4
2.3 MQTT dan Mosquitto MQTT <i>Broker</i>	4
2.4 Node-RED sebagai <i>Backend</i> IoT	5
2.5 <i>Dashboard Monitoring</i> Grafana.....	5
2.6 <i>Sistem Monitoring dan Early Warning</i>	5
2.7 Studi Literatur Proyek Serupa	6
C. ARSITEKTUR SISTEM	7
3.1 Arsitektur Sistem Keseluruhan.....	7
3.2 Arsitektur Infrastruktur Server	8
3.3 Arsitektur <i>Frontend–Backend</i>	9
D. IMPLEMENTASI INFRASTRUKTUR SERVER	11
4.1 Spesifikasi	11
4.2 Instalasi dan Konfigurasi Server	11
4.3 <i>Deployment Backend</i>	11
4.4 <i>Deployment Frontend Dashboard</i>	12
E. Implementasi <i>Frontend dan Backend</i>	13
5.1 Implementasi <i>Backend</i>	13
5.2 Implementasi <i>Frontend</i>	13
5.3 Integrasi IoT dengan <i>Backend</i>	14

5.4 Integrasi <i>Backend</i> dengan <i>Frontend</i>	14
F. Fungsionalitas <i>Dashboard</i>	16
6.1 <i>Dashboard Monitoring</i>	16
6.2 Visualisasi Data.....	16
6.3 Logging dan Histori Data.....	16
G. PENUTUP	17
7.1 Kesimpulan.....	17
7.2 Saran.....	18
DAFTAR PUSTAKA	19

DAFTAR GAMBAR

Gambar 1. Arsitektur sistem keseluruhan	7
Gambar 2. Arsitektur Backend-Frontend.....	9
Gambar 3. Dashboard Grafana	14
Gambar 4. Dashboard Node-RED	14

DAFTAR TABEL

Table 1. Spesifikasi VPS.....	11
-------------------------------	----

A. PENDAHULUAN

1.1 Latar Belakang

Perkembangan transformasi digital pada sektor perbankan menyebabkan ketergantungan yang semakin tinggi terhadap infrastruktur teknologi informasi. Berbagai layanan seperti data center, mesin ATM, jaringan komunikasi, serta sistem keamanan digital harus beroperasi secara terus-menerus untuk menjamin ketersediaan layanan kepada nasabah. Gangguan pada salah satu komponen infrastruktur tersebut dapat mengakibatkan penurunan kualitas layanan, kerugian operasional, hingga menurunkan kepercayaan masyarakat terhadap institusi perbankan. Oleh karena itu, ketahanan digital (*digital resilience*) menjadi salah satu aspek penting yang harus diperhatikan oleh industri perbankan modern [1].

Seiring berkembangnya teknologi *Internet of Things* (IoT), proses pemantauan infrastruktur dapat dilakukan secara *real-time* melalui pemanfaatan berbagai sensor yang mampu mendeteksi kondisi lingkungan maupun aktivitas perangkat. Teknologi IoT memungkinkan perangkat untuk mengumpulkan, mengirimkan, dan bertukar data melalui jaringan internet sehingga kondisi sistem dapat dipantau secara berkelanjutan [2]. Namun, data yang dihasilkan oleh perangkat IoT tidak akan memberikan manfaat optimal tanpa adanya sistem server yang mampu menerima, menyimpan, mengolah, serta menyajikan data tersebut menjadi informasi yang dapat digunakan dalam pengambilan keputusan [3].

Teknologi server memiliki peran penting sebagai pusat pengelolaan data dalam sistem *monitoring* modern. Server berfungsi sebagai media komunikasi antara perangkat IoT dan pengguna, sekaligus menjalankan berbagai layanan seperti *broker* MQTT, sistem pemrosesan data, *dashboard* monitoring, penyimpanan data historis, serta mekanisme notifikasi dini. Arsitektur berbasis *cloud* menggunakan Virtual Private Server (VPS) memberikan fleksibilitas yang tinggi karena memungkinkan layanan dapat diakses dari berbagai lokasi melalui jaringan internet dengan biaya yang relatif efisien dibandingkan infrastruktur fisik konvensional [4].

Dalam sistem IoT, protokol MQTT (Message Queuing Telemetry Transport) banyak digunakan karena memiliki karakteristik ringan, efisien, dan sesuai untuk komunikasi data *real-time* antar perangkat yang memiliki keterbatasan sumber daya [5]. MQTT menggunakan model *publish-subscribe* yang memungkinkan pertukaran data berlangsung secara cepat melalui *broker* sebagai perantara komunikasi. Selain itu, *platform* Node-RED banyak dimanfaatkan sebagai *backend system* karena mendukung pengembangan berbasis flow yang memudahkan integrasi antara perangkat IoT, basis data, dan layanan notifikasi [6].

Pada proyek ini dikembangkan sebuah sistem *Early Warning* Gangguan Infrastruktur Bank Berbasis IoT yang memanfaatkan VPS sebagai pusat layanan sistem. Server digunakan untuk menjalankan Mosquitto MQTT *Broker* sebagai media pertukaran

data, sementara Node-RED sebagai *backend system*, dan Hadoop sebagai *platform* pendukung dijalankan pada *Virtual Machine* lokal.

Melalui implementasi teknologi server tersebut, diharapkan sistem mampu mendukung proses pemantauan kondisi infrastruktur perbankan secara *real-time*, meningkatkan kecepatan deteksi gangguan, serta membantu administrator dalam mengambil tindakan yang diperlukan. Dengan demikian, teknologi server menjadi komponen utama yang menentukan keberhasilan sistem *early warning* dalam mendukung keamanan dan keandalan infrastruktur perbankan digital.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah pada penelitian ini adalah sebagai berikut:

1. Bagaimana merancang infrastruktur server yang mampu mendukung sistem *monitoring* berbasis IoT secara *real-time*?
2. Bagaimana mengimplementasikan layanan server menggunakan VPS sebagai pusat pengolahan dan penyimpanan data?
3. Bagaimana mengintegrasikan Mosquitto MQTT *Broker*, Node-RED, Hadoop, dan Grafana dalam satu sistem server yang terpusat?
4. Bagaimana menampilkan hasil *monitoring* perangkat IoT melalui *dashboard* yang mudah diakses dan dipahami oleh pengguna?
5. Bagaimana server dapat mendukung mekanisme *early warning* terhadap gangguan infrastruktur perbankan?

1.3 Tujuan

Tujuan dari pelaksanaan proyek pada mata kuliah Teknologi Server adalah:

1. Merancang dan membangun infrastruktur server berbasis VPS untuk mendukung sistem *monitoring* IoT.
2. Mengimplementasikan layanan Mosquitto MQTT *Broker* sebagai media komunikasi data antara perangkat IoT dan server.
3. Mengembangkan *backend system* menggunakan Node-RED untuk memproses data yang diterima dari perangkat IoT.
4. Membangun *dashboard monitoring* menggunakan Grafana untuk menampilkan data secara *real-time*.
5. Mengintegrasikan seluruh layanan server sehingga mampu mendukung sistem *early warning* pada infrastruktur perbankan.

1.4 Manfaat

Adapun manfaat yang diharapkan dari proyek ini adalah:

1. Bagi Pengguna Sistem

- a. Memudahkan proses pemantauan kondisi infrastruktur perbankan secara *real-time*.
- b. Membantu administrator memperoleh informasi gangguan secara lebih cepat.
- c. Mendukung pengambilan keputusan melalui penyajian data yang terstruktur dan mudah dipahami.

2. Bagi Institusi Perbankan

- a. Meningkatkan keandalan sistem *monitoring* infrastruktur.
- b. Mengurangi risiko keterlambatan penanganan gangguan.
- c. Mendukung penerapan konsep *digital resilience* dalam pengelolaan infrastruktur teknologi informasi.

3. Bagi Mahasiswa

- a. Menambah pengalaman dalam implementasi teknologi server berbasis *cloud*.
- b. Meningkatkan pemahaman mengenai integrasi VPS, MQTT *Broker*, Node-RED, Hadoop, dan Grafana.
- c. Menjadi sarana penerapan teori teknologi server dalam studi kasus nyata.

1.5 Batasan Masalah

Agar pembahasan lebih terarah, penelitian ini dibatasi pada beberapa hal berikut:

1. Sistem server diimplementasikan menggunakan Virtual Private Server (VPS).
2. Layanan yang digunakan pada server meliputi Mosquitto MQTT *Broker*, sementara layanan yang dijalankan pada *Virtual Machine* lokal meliputi Node-RED, Hadoop, dan Grafana.
3. Data yang diproses berasal dari perangkat IoT berupa sensor suhu, sensor getaran, dan sensor pintu.
4. *Dashboard monitoring* hanya menampilkan data hasil pemantauan dan status perangkat secara *real-time*.
5. Sistem notifikasi difokuskan pada mekanisme *early warning* terhadap kondisi abnormal yang terdeteksi dari data sensor.
6. Penelitian tidak mencakup integrasi dengan sistem core banking maupun sistem transaksi perbankan.
7. Aspek keamanan server yang dibahas terbatas pada konfigurasi dasar yang mendukung operasional sistem *monitoring*.

B. TINJAUAN PUSTAKA

2.1 Teknologi Server

Server merupakan sistem komputer yang menyediakan layanan, sumber daya, dan data kepada perangkat lain melalui jaringan. Dalam arsitektur client-server, server berfungsi sebagai pusat pengolahan data, penyimpanan informasi, serta penyedia layanan yang dapat diakses oleh banyak pengguna secara bersamaan [12].

Pada sistem *Internet of Things* (IoT), server memiliki peran penting sebagai pusat penerimaan data dari perangkat sensor, pengolahan data, penyimpanan data historis, serta penyedia antarmuka *monitoring* bagi pengguna [3]. Server juga berfungsi menjalankan berbagai layanan seperti database, *broker* komunikasi, *dashboard* monitoring, dan sistem notifikasi.

Dalam proyek ini, server digunakan sebagai pusat integrasi seluruh komponen sistem. Data yang dikirim oleh perangkat IoT diterima oleh server melalui *MQTT Broker*, kemudian diproses menggunakan Node-RED dan ditampilkan melalui *dashboard monitoring* berbasis Grafana.

2.2 Virtual Private Server (VPS)

Virtual Private Server (VPS) merupakan teknologi virtualisasi yang memungkinkan satu server fisik dibagi menjadi beberapa server virtual yang bekerja secara independen [4]. Setiap VPS memiliki sistem operasi, alokasi sumber daya, serta konfigurasi jaringan yang dapat dikelola secara terpisah.

Penggunaan VPS memberikan berbagai keuntungan, seperti fleksibilitas konfigurasi, kemudahan akses melalui internet, biaya operasional yang lebih rendah, serta kemampuan untuk menjalankan berbagai layanan dalam satu lingkungan server [4].

Pada proyek ini, VPS digunakan sebagai server pusat yang menjalankan layanan Mosquitto *MQTT Broker*, Node-RED, Hadoop, dan Grafana. Penggunaan VPS memungkinkan sistem *monitoring* dapat diakses secara *real-time* dari berbagai lokasi selama terhubung ke internet.

2.3 MQTT dan Mosquitto MQTT Broker

Message Queuing Telemetry Transport (MQTT) merupakan protokol komunikasi ringan yang dirancang untuk perangkat dengan keterbatasan sumber daya dan jaringan dengan *bandwidth* rendah [5]. MQTT menggunakan pola komunikasi *publish-subscribe* yang terdiri dari *publisher*, *broker*, dan *subscriber*.

Dalam mekanisme MQTT, perangkat IoT mengirimkan data ke *broker* melalui proses *publish*. Selanjutnya *broker* mendistribusikan data tersebut kepada *subscriber* yang membutuhkan informasi tersebut [13].

Mosquitto merupakan salah satu implementasi *MQTT Broker* yang bersifat *open source* dan banyak digunakan pada sistem IoT karena ringan, stabil, serta mudah

Pada proyek ini, Mosquitto MQTT *Broker* digunakan sebagai media komunikasi antara perangkat ESP32 dengan *backend* server yang dijalankan menggunakan Node-RED.

2.4 Node-RED sebagai *Backend* IoT

Node-RED merupakan *platform* pengembangan aplikasi berbasis flow yang dikembangkan oleh IBM untuk mendukung integrasi perangkat *Internet of Things* [6]. Node-RED menyediakan antarmuka visual berbasis web yang memungkinkan pengguna membangun alur pemrosesan data tanpa harus menulis kode secara kompleks.

Node-RED mendukung berbagai protokol komunikasi seperti MQTT, HTTP, TCP/IP, dan *WebSocket* sehingga banyak digunakan dalam pengembangan sistem *monitoring* berbasis IoT [6].

Dalam proyek ini, Node-RED berfungsi sebagai *backend system* yang menerima data dari MQTT *Broker*, melakukan pengolahan data, menentukan kondisi normal maupun abnormal, serta mengirimkan notifikasi peringatan dini kepada administrator.

2.5 Dashboard Monitoring Grafana

Grafana merupakan *platform* visualisasi data *open source* yang digunakan untuk membuat *dashboard monitoring* secara *real-time* [8]. Grafana mampu menampilkan data dalam berbagai bentuk visualisasi seperti grafik, tabel, *gauge*, maupun indikator status.

Dashboard monitoring memberikan kemudahan bagi administrator untuk memantau kondisi sistem secara terpusat. Informasi yang disajikan secara visual memungkinkan proses identifikasi gangguan dilakukan lebih cepat dibandingkan pemantauan manual [22].

Dalam proyek ini, Grafana digunakan untuk menampilkan data sensor suhu, sensor getaran, dan sensor pintu yang dikirimkan oleh perangkat IoT melalui server sehingga kondisi infrastruktur dapat dipantau secara *real-time*.

2.6 Sistem Monitoring dan *Early Warning*

Sistem *monitoring* merupakan mekanisme pengawasan yang dilakukan secara berkelanjutan terhadap suatu objek atau proses untuk memperoleh informasi kondisi terkini [25]. Sementara itu, sistem *early warning* merupakan sistem yang dirancang untuk memberikan peringatan dini ketika terdeteksi kondisi yang berpotensi menimbulkan gangguan.

Implementasi sistem *early warning* pada infrastruktur teknologi informasi bertujuan untuk mengurangi risiko kerusakan sistem melalui deteksi dini terhadap kondisi abnormal. Sistem ini umumnya memanfaatkan sensor sebagai sumber data, server sebagai pusat pengolahan data, serta *dashboard* dan notifikasi sebagai media penyampaian informasi kepada administrator [17].

Pada proyek ini, sistem *early warning* diterapkan melalui integrasi perangkat IoT, MQTT *Broker*, Node-RED, dan Grafana sehingga administrator dapat memperoleh informasi secara cepat ketika terjadi gangguan pada infrastruktur perbankan.

2.7 Studi Literatur Proyek Serupa

Penelitian yang dilakukan oleh Naik [13] menunjukkan bahwa MQTT memiliki performa yang lebih efisien dibandingkan HTTP pada lingkungan *Internet of Things* karena penggunaan *bandwidth* yang lebih rendah dan latensi komunikasi yang lebih kecil.

Hanes et al. [2] menjelaskan bahwa integrasi IoT dengan *cloud* server mampu meningkatkan efektivitas proses *monitoring* melalui pengelolaan data secara terpusat serta kemudahan akses terhadap informasi secara *real-time*.

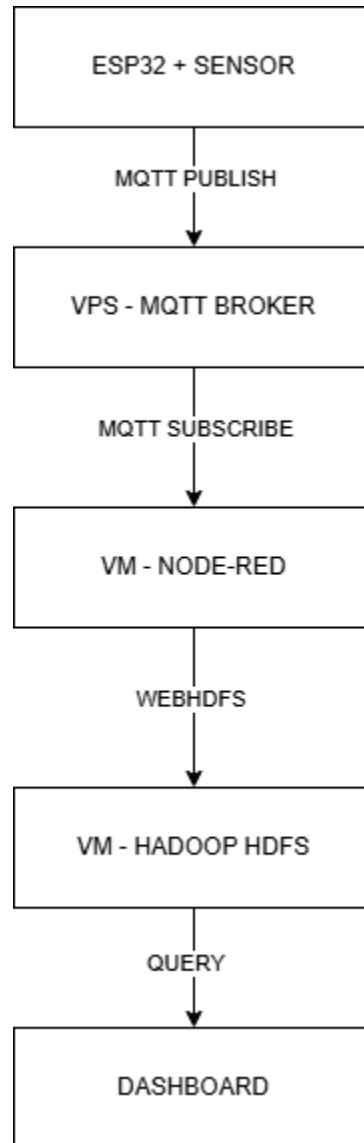
Penelitian oleh Keller [22] mengenai implementasi Grafana sebagai *platform monitoring* menunjukkan bahwa visualisasi data secara *real-time* dapat membantu administrator mengidentifikasi gangguan lebih cepat dan meningkatkan efektivitas pengambilan keputusan.

Berdasarkan berbagai penelitian tersebut dapat disimpulkan bahwa kombinasi VPS, MQTT *Broker*, Node-RED, dan Grafana merupakan pendekatan yang sesuai untuk membangun sistem *monitoring* dan *early warning* berbasis IoT yang terintegrasi.

C. ARSITEKTUR SISTEM

3.1 Arsitektur Sistem Keseluruhan

Sistem ini dirancang sebagai *platform monitoring* dan keamanan bank berbasis IoT yang mengintegrasikan perangkat keras sensor dengan infrastruktur pengolahan data terdistribusi. Secara keseluruhan, arsitektur sistem terbagi menjadi empat lapisan utama yang saling terhubung, yaitu lapisan perangkat IoT, lapisan komunikasi, lapisan pemrosesan dan penyimpanan data, serta lapisan visualisasi dan analisis.



Gambar 1. Arsitektur sistem keseluruhan

1. Lapisan perangkat IoT

Lapisan ini terdiri dari mikrokontroler ESP32 yang dilengkapi dengan sensor getaran, sensor pintu magnetik, sensor dht11, dan modul RFID yang bertugas

membaca kondisi fisik secara real-time dan mengirimkan data ke server melalui protokol MQTT.

2. Lapisan komunikasi

Lapisan ini menggunakan protokol MQTT (Message Queuing Telemetry Transport) dengan *broker* Mosquitto yang berjalan pada VPS. MQTT dipilih karena sifatnya yang ringan, efisien dalam penggunaan *bandwidth*, dan mendukung pola *publish-subscribe* yang cocok untuk perangkat IoT dengan sumber daya terbatas.

3. Lapisan pemrosesan dan penyimpanan

Lapisan ini terdiri dari Hadoop yang berjalan pada virtual machine (VM) lokal. Data yang diterima dari *broker* MQTT diteruskan oleh Node-RED melalui webHDFS API ke sistem file terdistribusi HDFS, dengan dua direktori utama, yaitu */data-realtime* untuk data langsung dan */data-backup* untuk arsip.

4. Lapisan visualisasi dan analisis

Lapisan ini menggunakan Node-RED sebagai *dashboard monitoring real-time* untuk analisis data historis menggunakan query SQL-like pada data yang tersimpan di HDFS. Selain Node-RED juga menggunakan Grafana untuk visualisasi data.

3.2 Arsitektur Infrastruktur Server

Infrastruktur server pada sistem ini mengadopsi pendekatan arsitektur hybrid yang secara strategis memisahkan antara fungsi *broker* komunikasi dengan unit penyimpanan serta pengolahan data guna mencapai efisiensi penggunaan sumber daya yang optimal.

1. Virtual Private Server (VPS) Kampus

VPS dengan spesifikasi RAM 8 GB yang disediakan oleh institusi kampus berperan sebagai titik masuk data dari perangkat IoT. Layanan yang berjalan di VPS adalah Mosquitto MQTT *Broker* yang akan menerima data dari ESP32 pada port 1883, melalui konfigurasi listener 1883 0.0.0.0 dan *allow_anonymous true*. VPS dapat diakses melalui IP lokal kampus, yaitu 10.33.109.55.

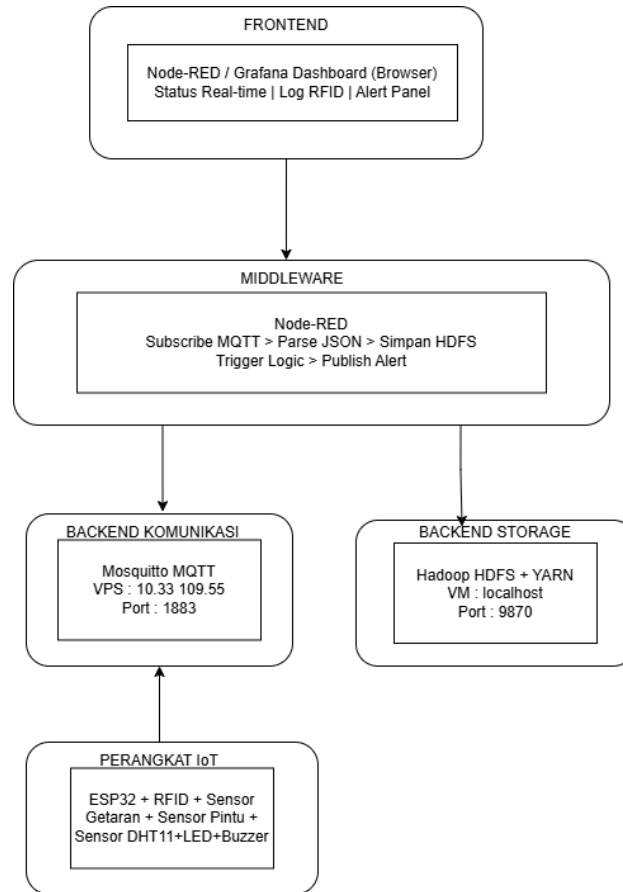
2. Virtual Machine (VM) Lokal

VM lokal berperan sebagai tempat pemrosesan dan penyimpanan data besar. Layanan yang berjalan di VM adalah Hadoop dengan mode *pseudo-distributed* dengan NameNode dan DataNode pada mesin yang sama, kemudian terdapat Node-RED sebagai *middleware* visual yang menghubungkan MQTT *broker* di VPS dengan HDFS pada VM, dan terakhir terdapat YARN Resource Manager yang digunakan untuk mengelola sumber daya komputasi untuk job MapReduce.

Koneksi antara VPS dan VM memanfaatkan jaringan kampus yang sama. Node-RED di VM melakukan subscribe ke *broker* MQTT di VPS menggunakan IP lokal kampus, kemudian menyimpan data ke HDFS melalui WebHDFS API pada localhost.

3.3 Arsitektur *Frontend-Backend*

Struktur sistem ini mengadopsi pola *frontend-backend* yang disusun melalui arsitektur berlapis, yang secara sistematis memisahkan komponen antarmuka pengguna, mekanisme logika pemrosesan, serta infrastruktur penyimpanan data. Arsitektur ini terintegrasi melalui protokol MQTT.



Gambar 2. Arsitektur Backend-Frontend

1. *Backend*

Backend sistem bertanggung jawab dalam menerima data dari sistem IoT, melakukan pemrosesan data, berinteraksi dengan *database*, serta menyediakan layanan yang dapat diakses oleh *frontend*. *Backend* sistem terdiri dari dua komponen utama, yaitu MQTT Broker (Mosquitto) yang berfungsi sebagai *backend* komunikasi real-time. Broker akan menerima pesan dari ESP32 dalam format JSON melalui topik-topik yang terstruktur pada tiap sensor, seperti vibradoor/getaran untuk menerima data dari sensor getaran. Sistem kedua adalah HDFS + YARN yang berfungsi untuk menyimpan dan memproses data dalam ukuran besar. Setiap data yang masuk dari MQTT Broker, data akan disimpan dalam format JSON pada direktori HDFS dan dapat dianalisis menggunakan MapReduce atau Hive untuk menghasilkan statistik.

2. *Middleware*

Middleware sistem bertugas untuk menghubungkan antara *backend* sistem dan *frontend* sistem. Komponen *middleware* pada sistem ini adalah Node-RED. Pada sistem ini tugas yang dilakukan Node-RED adalah subscribe ke semua topik MQTT dari VPS, *parsing* dan transformasi data JSON, menyimpan data ke HDFS via WebHDFS, juga sebagai logika *trigger*.

3. *Frontend*

Frontend sistem berfungsi sebagai antarmuka pengguna yang menampilkan informasi *monitoring* sensor dalam bentuk tabel, grafik, maupun indikator kondisi perangkat. Pada sistem ini, frontend sistem menggunakan *dashboard* bawaan Node-RED yang dapat diakses melalui browser, dan grafana. Dashboard Node-RED menampilkan status sensor pintu dan getaran secara *real-time*, log scan RFID beserta status ilegal maupun legal, nilai suhu dan kelembaban yang dideteksi secara real-time, rata - rata dari suhu dan kelembaban. Sama seperti *dashboard Node-RED*, *dashboard* grafana menampilkan status sensor pintu dan getaran secara *real-time*, log scan RFID beserta status ilegal maupun legal, nilai suhu dan kelembaban yang dideteksi secara real-time, rata - rata dari suhu dan kelembaban.

D. IMPLEMENTASI INFRASTRUKTUR SERVER

4.1 Spesifikasi

Implementasi sistem dilakukan pada VPS yang difasilitasi oleh kampus yang berfungsi sebagai host utama. Berikut spesifikasi dari VPS yang digunakan :

Table 1. Spesifikasi VPS

CPU	4 CORE
RAM	8 GB
DISK	50 – 100 NVMe SSD
OS	Ubuntu 22.04 LTS

Selain host server, dibuat beberapa VM untuk menjalankan layanan secara terpisah guna meningkatkan keamanan dan kemudahan administrasi server.

4.2 Instalasi dan Konfigurasi Server

Tahap implementasi diawali dengan instalasi server. Proses ini sudah dilakukan oleh pihak kampus, sehingga mahasiswa tidak perlu lagi melakukan konfigurasi dasar untuk server. Instalasi layanan yang dilakukan di server adalah *MQTT Broker*. Konfigurasi layanan ini bertujuan untuk menerima koneksi dari perangkat IoT melalui port komunikasi yang telah ditentukan.

Isi dari file *mqtt broker listener 1883 0.0.0.0* untuk memastikan broker dapat menerima koneksi dari semua antarmuka jaringan, tidak hanya localhost. Direktif **allow_anonymous true** mengizinkan koneksi tanpa autentikasi untuk kemudahan development. Selanjutnya dilakukan konfigurasi virtualisasi dan pembuatan VM untuk menjalankan layanan Hadoop, Node-RED secara terpisah dari VPS.

4.3 Deployment Backend

Backend sistem terdiri dari dua komponen yang berjalan secara independen namun saling terhubung. Backend sistem pertama yaitu Mosquitto di *deploy* di VPS, dan dikonfigurasi untuk berjalan otomatis saat sistem boot menggunakan perintah **sudo systemctl enable mosquitto**. Topik MQTT yang didaftarkan untuk sistem ini adalah *bank/vault/vibration* untuk menerima data sensor getaran, *bank/vault/door* untuk menerima data status pintu apakah terbuka/tertutup, *bank/vault/access* untuk data scan kartu RFID, *bank/dc/sensor1/suhu* untuk menerima data suhu setiap sensor, *bank/dc/sensor1/hum* untuk menerima data kelembaban setiap sensor.

Sistem kedua adalah *deploy* Hadoop yang dijalankan setiap sesi kerja pada VM. *Backend* dikonfigurasi untuk menerima data dari *MQTT Broker* yang berjalan pada VPS melalui topik-topik sensor yang telah disebutkan sebelumnya. Data tersebut kemudian diproses oleh Node-RED melalui *flow* yang menggabungkan seluruh parameter sensor menjadi satu objek JSON, menambahkan *timestamp*, dan menyimpannya ke file sementara

/tmp/sensor_realtime.json. File ini kemudian diunggah ke HDFS pada folder /data-realtime setiap 5 menit melalui mekanisme *scheduler* bawaan Node-RED. Selain itu, sistem *backup* berlapis juga diterapkan, yaitu HDFS Snapshot otomatis setiap 1 jam untuk perlindungan data jangka pendek, serta *cold backup* mingguan setiap hari Jumat pukul 00.00 yang menyalin data dari /data-realtime ke /data-backup dengan format nama backup_bulan_tahun.json sebagai arsip mingguan. Seluruh proses *backup* dan upload dapat dipantau melalui node debug pada Node-RED yang menampilkan notifikasi keberhasilan atau kegagalan operasi.

4.4 Deployment Frontend Dashboard

Dashboard dibangun menggunakan Node-RED Dashboard (node-red dashboard) yang menyediakan antarmuka web berbasis komponen *gauge*, teks, dan indikator status. Dashboard dapat diakses melalui browser pada alamat <http://192.168.31.129:1880/ui> pada VM. Flow Node-RED di-*deploy* melalui antarmuka editor Node-RED dengan mengimpor file JSON konfigurasi flow, kemudian menekan tombol *deploy*. Seluruh konfigurasi tersimpan di direktori yang sudah ditentukan, sehingga *persistent* antar sesi.

E. Implementasi *Frontend* dan *Backend*

5.1 Implementasi *Backend*

Layanan *backend* berfungsi sebagai pusat pengelolaan data dan komunikasi antar komponen sistem. Implementasi backend dilakukan pada beberapa lapisan, yaitu :

1. Lapisan komunikasi *real-time* (mosquitto)

Mosquitto MQTT *Broker* berfungsi sebagai pengirim pesan utama. Setiap data yang dikirim ESP32 diterima oleh broker dan segera diteruskan ke semua client yang subscribe pada topik yang relevan. Arsitektur *publish-subscribe* ini memungkinkan beberapa *subscriber* menerima data yang sama secara bersamaan tanpa mengubah kode pengirim.

2. Lapisan pemrosesan data

Setiap topik MQTT memiliki function node tersendiri yang bertanggung jawab memvalidasi, membersihkan, dan menyimpan data ke *flow context*. Logika pemrosesan utama terdapat pada node yang menghitung rata-rata yang mengagregasi semua data sensor menjadi satu *payload* terpadu sebelum diteruskan ke *dashboard* HDFS. Data sensor yang sudah diterima node-RED akan ditulis ke HDFS dalam format JSON untuk memudahkan pemrosesan *batch*.

5.2 Implementasi *Frontend*

Frontend diimplementasikan sebagai layanan yang dapat diakses pengguna menggunakan Node-RED *Dashboard* yang menghasilkan halaman web responsif secara otomatis berdasarkan konfigurasi node di editor Node-RED. Komponen UI yang digunakan adalah *ui_gauge* untuk menampilkan nilai numerik suhu dan kelembaban dalam bentuk gauge setengah lingkaran dengan rentang warna hijau menunjukkan normal, dan merah menunjukkan kritis. Tampilan ini digunakan untuk 6 sensor suhu, 6 sensor getaran, dan 1 gauge rata-rata suhu data center. Komponen UI selanjutnya adalah *ui_text* yang menampilkan status teks dinamis untuk sensor getaran, status pintu, informasi RFID, dan panel *alert*. Setiap teks diperbarui secara *real-time* setiap kali data baru masuk dari MQTT.

Layout dashboard dibagi menjadi beberapa group yang ditampilkan seperti gambar berikut :



Gambar 3. Dashboard Grafana



Gambar 4. Dashboard Node-RED

5.3 Integrasi IoT dengan *Backend*

Integrasi antara ESP32 dan *backend* dilakukan melalui protokol MQTT dengan library PubSubClient pada Arduino framework. ESP32 bertindak sebagai MQTT *publisher* yang mengirimkan data sensor setiap kali ada perubahan kondisi, bukan secara periodik untuk menghemat *bandwidth*. Alur integrasi dimulai dengan ESP32 terhubung ke WiFi kampus, kemudian membentuk koneksi ke *broker* Mosquitto, data dari sensor dibaca dan diformat menjadi JSON, selanjutnya *payload* dikirimkan ke topik yang sesuai, dan koneksi akan tetap dijaga dan akan *reconnect* otomatis jika terputus. ESP32 juga subscribe ke topik vibradoor/alert untuk menerima perintah balik dari sistem, yang memungkinkan implementasi *feedback loop* jika diperlukan di masa mendatang.

5.4 Integrasi *Backend* dengan *Frontend*

Node-RED berfungsi sebagai jembatan antara *backend* dan *frontend* pada *dashboard* awal yang dibangun menggunakan Node-RED Dashboard. Pada *dashboard* ini, setiap perubahan data di MQTT langsung memperbarui tampilan *dashboard* melalui

WebSocket yang dikelola oleh Node-RED secara internal, sehingga latensi tampilan mendekati *real-time*.

Sementara itu, untuk *dashboard* final yang dibangun menggunakan Grafana, mekanisme integrasi berbeda. Grafana terhubung langsung ke broker MQTT melalui *plugin* MQTT *datasource* yang melakukan *subscribe* ke topik-topik yang sama dengan Node-RED. Dengan demikian, data mengalir langsung dari broker ke Grafana tanpa melalui Node-RED, sehingga dashboard tetap dapat menampilkan data secara *real-time* meskipun tidak bergantung pada *WebSocket* Node-RED.

F. Fungsionalitas *Dashboard*

6.1 *Dashboard Monitoring*

Dashboard monitoring menyediakan tampilan terpusat untuk memantau kondisi fisik dan keamanan data center secara *real-time*. *Dashboard* dapat diakses melalui browser pada VM. tampilan dashboard menampilkan kondisi 6 rak server secara bersamaan dalam satu halaman, serta sensor lainnya, sehingga operator dapat memantau seluruh ruangan sekaligus.

6.2 Visualisasi Data

Data divisualisasikan dalam bentuk *gauge* real time yang digunakan untuk menampilkan nilai dari sensor dht11 berbentuk setengah lingkaran, dan teks status dinamis yang digunakan untuk menampilkan status dari kondisi getaran, pintu, dan RFID yang ditampilkan dalam format teks deskriptif untuk kemudahan pembacaan.

6.3 Logging dan Histori Data

1. Logging Data *Real-Time*

Setiap data yang masuk dari MQTT dicatat ke file yang sudah ditentukan dalam format JSON Lines. File ini berfungsi sebagai *buffer* sementara sebelum diunggah ke HDFS.

2. Penyimpanan Histori di HDFS

Data histori dikelola dalam dua tingkat, yaitu *hot storage* yang menyimpan data sensor terkini yang diunggah setiap 5 menit, sehingga mudah diakses untuk dianalisis jarak pendek. Selanjutnya terdapat *cold storage* yang menyimpan *snapshot* mingguan yang dibuat setiap jumat pukul 00.00. Data ini diperuntukkan untuk analisis tren jangka panjang dan audit. Snapshot HDFS dibuat setiap 1 jam untuk pemulihan data jika terjadi kegagalan sistem.

G. PENUTUP

7.1 Kesimpulan

Berdasarkan hasil implementasi sistem *early warning*, dapat ditarik beberapa kesimpulan seperti berikut :

1. Integrasi IoT dengan infrastruktur server berhasil diimplementasikan
Sistem berhasil mengintegrasikan perangkat keras ESP32 dengan sensor getaran SW-420, sensor pintu reed switch, dan modul RFID MFRC522 ke dalam infrastruktur server yang terdiri dari VPS kampus dan VM lokal. Data dari ketiga sensor berhasil dikirimkan secara *real-time* menggunakan protokol MQTT melalui *broker* Mosquitto yang berjalan di VPS dengan IP lokal kampus 10.33.109.55.
2. Pipeline data dari IoT hingga penyimpanan *big data* berfungsi dengan baik
Data sensor yang masuk melalui MQTT *broker* berhasil diteruskan oleh Node-RED ke sistem penyimpanan terdistribusi Apache Hadoop HDFS. Pipeline mencakup penyimpanan sementara di `/tmp/sensor_realtime.json`, unggahan berkala setiap 5 menit ke direktori `/data-realtime`, serta backup mingguan otomatis ke direktori `/data-backup` setiap Jumat pukul 00.00.
3. *Dashboard monitoring real-time* berhasil dibangun dalam dua tahap dan berfungsi
Node-RED *Dashboard* digunakan sebagai prototipe awal untuk memverifikasi aliran data dan logika *alert*, menampilkan enam sensor suhu dan kelembaban, status getaran, status pintu, akses RFID, serta panel *alert* kombinasi secara *real-time* melalui *WebSocket* internal Node-RED, dapat diakses di `http://192.168.31.129:1880/ui`. Selanjutnya, Grafana diimplementasikan sebagai *dashboard* final yang terhubung langsung ke broker MQTT melalui plugin MQTT *datasource*. Dashboard Grafana dapat diakses di `http://192.168.31.129:3000` dengan visualisasi lebih lengkap untuk rata-rata suhu, rata-rata kelembaban, status getaran, status pintu, akses RFID, dan *alert* gabungan.
4. Sistem notifikasi dan alarm berfungsi secara otomatis
Sistem berhasil mengimplementasikan dua lapis notifikasi: alert kombinasi berbasis logika di Node-RED yang mendeteksi kondisi berbahaya secara bersamaan (pintu terbuka dan kartu RFID ilegal), serta aktuator buzzer dan LED otomatis tanpa interaksi manual.
5. Kendala keterbatasan sumber daya berhasil diatasi dengan arsitektur hybrid
Keterbatasan RAM VPS kampus diatasi dengan menerapkan arsitektur hybrid: VPS hanya menjalankan layanan MQTT *broker* yang ringan, sementara Apache Hadoop dan Node-RED dipindahkan ke VM lokal. Pendekatan ini terbukti efektif dalam menjaga stabilitas sistem tanpa mengorbankan fungsionalitas.
6. Visualisasi historis data center dapat dilakukan melalui Node-RED dan Grafana
Integrasi Node-RED sebagai API endpoint dengan Grafana memungkinkan visualisasi data historis dalam bentuk grafik *time-series*, sehingga operator dapat

memantau tren suhu dan kelembaban data center dalam rentang waktu tertentu untuk keperluan analisis dan audit.

7.2 Saran

Berdasarkan pengalaman implementasi dan keterbatasan yang ditemukan selama pengerjaan, berikut beberapa saran untuk pengembangan sistem ke depan :

1. Penerapan Autentikasi MQTT

Konfigurasi Mosquitto saat ini menggunakan `allow_anonymous true` yang memungkinkan siapapun terhubung ke *broker* tanpa autentikasi. Untuk lingkungan produksi, disarankan menerapkan autentikasi username dan password pada Mosquitto, serta mempertimbangkan penggunaan MQTT over TLS (port 8883) untuk enkripsi data yang dikirimkan sensor.

2. Penambahan sensor yang lebih lengkap

Sistem saat ini baru mengimplementasikan sensor getaran, pintu, dan RFID untuk aspek keamanan, serta sensor suhu dan kelembaban untuk *monitoring* lingkungan. Untuk monitoring data center yang lebih komprehensif, disarankan menambahkan sensor arus listrik untuk memantau konsumsi daya per rak, sensor asap dan detektor kebakaran, serta sensor kebocoran air di bawah raised floor.

3. Implementasi sistem manajemen pengguna

Dashboard saat ini dapat diakses oleh siapapun tanpa autentikasi. Disarankan menambahkan sistem manajemen pengguna dengan setidaknya dua level akses: administrator yang dapat melihat semua data dan mengubah konfigurasi, serta operator yang hanya dapat memantau dashboard tanpa mengubah pengaturan. Hal ini dapat diimplementasikan melalui fitur `adminAuth` bawaan Node-RED atau menggunakan reverse proxy seperti Nginx dengan autentikasi dasar.

4. Penerapan mekanisme failover dan high availability

Sistem saat ini bersifat single point of failure — jika VPS atau VM mati, seluruh sistem *monitoring* berhenti berfungsi. Untuk implementasi produksi yang membutuhkan keandalan tinggi, disarankan menerapkan cluster Hadoop dengan minimal tiga node, replikasi MQTT *broker* menggunakan bridge, serta mekanisme health check dan restart otomatis untuk semua layanan menggunakan systemd atau container orchestration seperti Docker Compose.

5. Integrasi sistem notifikasi

Notifikasi *alert* pada sistem saat ini terbatas pada tampilan visual di *dashboard* dan aktuator. Kondisi ini beresiko menyebabkan keterlambatan respons jika tidak ada operator yang memantau layar saat insiden terjadi. Untuk mengatasi hal tersebut, disarankan beberapa pengembangan seperti notifikasi via *email* maupun *WhatsApp*. Dengan integrasi multi-channel ini, sistem notifikasi tidak lagi bergantung pada keberadaan operator di depan layar, sehingga respons terhadap insiden keamanan dan lingkungan data center dapat dilakukan secara lebih cepat dan andal.

DAFTAR PUSTAKA

- [1] Otoritas Jasa Keuangan, *Panduan Resiliensi Digital bagi Bank Umum*. Jakarta, Indonesia: OJK, 2024.
- [2] D. Hanes, G. Salgueiro, P. Grossetete, R. Barton, and J. Henry, *IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things*. Indianapolis, IN, USA: Cisco Press, 2017.
- [3] T. Erl, *Cloud Computing: Concepts, Technology and Architecture*. Upper Saddle River, NJ, USA: Pearson Education, 2013.
- [4] R. Buyya, J. Broberg, and A. Goscinski, *Cloud Computing: Principles and Paradigms*. Hoboken, NJ, USA: Wiley, 2011.
- [5] A. Banks and R. Gupta, *MQTT Version 3.1.1*. OASIS Standard, 2014.
- [6] Node-RED Project, “Node-RED Documentation,” 2025. [Online]. Available: <https://nodered.org/docs/>. [Accessed: Jun. 2026].
- [7] Eclipse Foundation, “Eclipse Mosquitto Documentation,” 2025. [Online]. Available: <https://mosquitto.org/documentation/>. [Accessed: Jun. 2026].
- [8] Grafana Labs, “Grafana Documentation,” 2025. [Online]. Available: <https://grafana.com/docs/>. [Accessed: Jun. 2026].
- [9] Apache Software Foundation, “Apache Hadoop Documentation,” 2025. [Online]. Available: <https://hadoop.apache.org/docs/>. [Accessed: Jun. 2026].
- [10] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston, MA, USA: Pearson, 2011.
- [11] W. Stallings, *Network Security Essentials: Applications and Standards*, 7th ed. Boston, MA, USA: Pearson, 2022.
- [12] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [13] N. Naik, “Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP,” in *Proc. IEEE International Systems Engineering Symposium*, Vienna, Austria, 2017, pp. 1–7.
- [14] M. Minelli, M. Chambers, and A. Dhiraj, *Big Data, Big Analytics: Emerging Business Intelligence and Analytic Trends for Today's Businesses*. Hoboken, NJ, USA: Wiley, 2013.

- [15] K. Ashton, "That 'Internet of Things' Thing," *RFID Journal*, Jun. 2009.
- [16] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*, 5th ed. Cham, Switzerland: Springer, 2016.
- [17] R. Bejtlich, *The Practice of Network Security Monitoring: Understanding Incident Detection and Response*. San Francisco, CA, USA: No Starch Press, 2013.
- [18] S. Mackay, E. Wright, D. Reynders, and J. Park, *Practical Industrial Data Communications*. Oxford, U.K.: Newnes, 2004.
- [19] P. Horowitz and W. Hill, *The Art of Electronics*, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2015.
- [20] T. Wilmshurst, *Designing Embedded Systems with PIC Microcontrollers*. Oxford, U.K.: Newnes, 2010.
- [21] A. Bahga and V. Madiseti, *Internet of Things: A Hands-On Approach*. Atlanta, GA, USA: VPT, 2015.
- [22] Grafana Labs, "Introduction to Grafana," Grafana Documentation, 2025. [Online]. Available: <https://grafana.com/docs/grafana/latest/introduction/>. [Accessed: Jun. 2026].
- [23] J. Axelson, *Serial Port Complete*, 2nd ed. Madison, WI, USA: Lakeview Research, 2007.
- [24] E. O. Doebelin and D. N. Manik, *Measurement Systems: Application and Design*, 6th ed. New York, NY, USA: McGraw-Hill, 2011.
- [25] National Institute of Standards and Technology, Guide to Industrial Control Systems (ICS) Security, NIST Special Publication 800-82 Rev. 2, Gaithersburg, MD, USA, 2015.